

TP 1 Couche Transport

TP Réseaux Avancés M1 SIC - IA

Ilyas Bambrik

Table des matières



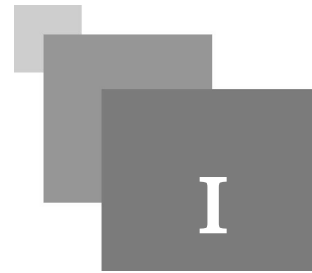
Objectifs	3
I - Exercice : netstat	4
II - Exercice : Multi-threading	9
III - Exercice : Interopérabilité Client / Serveur	11
IV - Analyse du trafic avec Wireshark	13
V - Exercice : Three way handshake et flags	15
VI - Exercice : Déploiement d'une application réseau simple	16

Objectifs

- Analyser l'état des connexions de processus;
- Analyser l'entête TCP avec Wireshark ;
- Travail à rendre : pour chaque question/étape prenez une capture d'écran *complète* (les captures d'écran partielles ne seront comptabilisés). *Le document à consulter doit être un fichier PDF avec les réponses et captures numérotées ;*
- *L'organisation du document à consulté est comptabilisée dans la note ;*
- Le TP sera consulté deux semaines après la publication de celui-ci ;

Les codes présentés dans ce support sont téléchargeables à partir du lien suivant
<https://drive.google.com/drive/u/0/folders/1XDPqzfoa8K7gOWcoOtbAqpXK7MYX4z9H>

Exercice : netstat



Lancez votre invité de commande (*CMD*) et exécutez la commande suivante pour afficher les connexions établies entre les applications dans votre machines et d'autres processus dans des machines distantes:

`netstat -n`

Remarque : Une connexion peut être établie entre deux applications qui fonctionnent dans la même machine.

Proto	Adresse locale	Adresse distante	État
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
TCP	127.0.0.1:61605	127.0.0.1:61562	TIME_WAIT
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
TCP	127.0.0.1:62562	127.0.0.1:5357	TIME_WAIT
TCP	192.168.1.2:62012	23.35.212.54:443	CLOSE_WAIT
TCP	192.168.1.2:62557	40.69.216.129:443	ESTABLISHED
TCP	192.168.1.2:62558	93.186.135.58:80	TIME_WAIT

Question 1

Afin d'analyser facilement le résultat de la commande, redirigez le résultat de netstat vers un fichier de sortie (par exemple : `netstat -n>FichierSortie.txt`)

- Exécutez le programme dans *Listing 1* (Serveur.py) avant de lancer le programme de *Listing 2* (Client .py). Les deux programmes doivent être lancés dans deux Shell différents.
- Exécutez la commande `netstat -n` à nouveau et vérifiez s'il y a un nouveau port ouvert correspondant au port de votre application Serveur (9500). La signification de option -n est :
-n : Affiche les adresses et les numéros de port sous forme numérique au lieu d'essayer de résoudre les noms des machines ;

```

1 import socket
2
3 SocketServeur = socket.socket()
4 port = 9500
5 # creation d'un socket en ecout sur l'interface 127.0.0.1, port=9500
6 SocketServeur.bind(("127.0.0.1", port))
7 SocketServeur.listen(5) # nombre de maximale de connexion qui peuvent etre mis
   en file d'attente
8
9 print( "Lancement serveur") # instruction d'affichage
10 while True: # boucle infinie
11     # accept bloque le programme en attendant une connexion d'un client (la
   reception du SYN)
12     ConnexionAUnClient, addrclient = SocketServeur.accept()
13     # une fois que la connexion est recue, accept renvoie l'adresse du client et
   un objet socket
14     # permettant l'envoi et la reception sur cette connexion
15     print("Connexion de la machine = ", addrclient)
16     MessageRec=""
17     while(MessageRec.strip().lower()!="fin"):# tanque le message recu est
   different de "fin"
18         MessageRec=ConnexionAUnClient.recv(1024) # recv permet de recevoir une
   sequence d'octets
19         print( MessageRec.decode())
20         # .decode() transforme une objet bytes (suite d'octets) en chaine de
   caracteres en
21
22     print( "Deconnexion de :",addrclient)
23     ConnexionAUnClient.close() # cloture la connexion (envoie FIN ACK au client)
24
25

```

Listing 1 Programme serveur python

```

1 import socket
2
3 ConnexionAUnServeur = socket.socket() # Creation de socket
4 host = "127.0.0.1" # adresse de la machine distante
5 port = 9500 # numero de port
6 print( "Console Client")
7 ConnexionAUnServeur.connect((host, port)) # connexion avec le serveur ( envoie
   flag SYN)
8 Message_A_Transmettre=""
9 while Message_A_Transmettre.lower()!="fin":
10     Message_A_Transmettre=input() # lecture depuis le clavier
11     ConnexionAUnServeur.send(Message_A_Transmettre.encode()) # transmettre le
   message vers le serveur
12     # .encode() transforme une chaine de caracteres en bytes (suite d'octets)
13 ConnexionAUnServeur.close() # cloture de la connexion avec le serveur (envoie
   flag FIN)
14 print( "Fin du programme")
15

```

Listing 2 Programme client

Invite de commandes

Connexions actives

Proto	Adresse locale	Adresse distante	État
TCP	127.0.0.1:9500	127.0.0.1:62631	ESTABLISHED
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
TCP	127.0.0.1:62628	127.0.0.1:62630	ESTABLISHED
TCP	127.0.0.1:62630	127.0.0.1:62628	ESTABLISHED
TCP	127.0.0.1:62631	127.0.0.1:9500	ESTABLISHED
TCP	192.168.1.2:62012	23.35.212.54:443	CLOSE_WAIT

- Quel est l'état de la connexion avec le serveur (voir la dernière colonne de l'affichage netstat)? Prenez une capture d'écran de l'affichage et encadrez le numéro de port du client.
- Transmettez un message du client pour le recevoir sur le serveur (écrivez un message dans la console Client et appuyez sur Enter).

Afin de terminer un processus occupant un numéro de port cible (comme notre serveur par exemple), exécutez les deux commandes suivantes :

- Trouvez le PID du processus occupant le numéro de port ciblé (dans ce cas 9500): `netstat -aon|findstr :9500` ;
- Terminer le processus avec le PID trouvé. Par exemple, pour terminer le processus 1700: `taskkill /PID 1700 /F` ;

La signification de option -a et -o (-aon est équivalente à -a -o -n) est :

- a : Affiche toutes les connexions et les ports d'écoute.
- o : Displays the owning process ID associated with each connection.

Question 2

- Lancez votre CMD en *mode administrateur* et exécutez la commande `netstat -n -b` pour voir les noms des processus correspondants à chaque connexion.

Proto	Adresse locale	Adresse distante	État
TCP	127.0.0.1:9500	127.0.0.1:62664	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:62628	127.0.0.1:62630	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:62630	127.0.0.1:62628	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:62664	127.0.0.1:9500	ESTABLISHED
[java.exe]			
TCP	192.168.1.2:62012	23.35.212.54:443	CLOSE_WAIT
[Video.UI.exe]			

- Transmettez un message *"Fin"* (chaîne de caractères Fin) de votre client active pour clôturer la connexion et ensuite lancez *netstat -nb* pour voir l'état de connexion entre client et serveur.

La signification de option -b (-nb est équivalente à -n -b) est :

- *-b* : Affiche le fichier exécutable impliqué dans la création de chaque connexion ou port d'écoute ;

Connexions actives

Proto	Adresse locale	Adresse distante	État
TCP	127.0.0.1:9500	127.0.0.1:62711	CLOSE_WAIT
[pythonw.exe]			
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
[javaw.exe]			
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
[scenari.exe]			
TCP	127.0.0.1:62708	127.0.0.1:62710	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:62710	127.0.0.1:62708	ESTABLISHED
[pythonw.exe]			
TCP	127.0.0.1:62711	127.0.0.1:9500	FIN_WAIT_2
[System]			

Question 3

- Lancez deux instances du client avec une seule instance du serveur. Transmettez ensuite des messages à partir des deux clients.
- Que remarquez-vous (est-ce que les messages des deux clients sont reçus)?

Exercice : Multi-threading

II

Le programme serveur précédant peut servir un seul client à la fois. Un serveur web par exemple doit servir plusieurs interlocuteurs simultanément. Ceci est réalisable grâce à la programmation *multi-threading*. Le programme python dans Listing 3 est un simple serveur multithread qui lance un processus chaque fois qu'un nouveau client est connecté.

Question

- Lancez le programme client (*Listing 2 du 1er exercice*) et puis le programme serveur multi-thread (*Listing 3*) ;
- Lancez un nouveau client sans fermer le 1er client et transmettez des messages depuis le 1er et 2em client (vérifiez que les messages des deux clients sont reçus par le serveur multithread python). Le deuxième client doit être lancé dans un Shell séparé.
- Exécutez la commande *netstat -n*;
- *Que remarquez-vous ?*

```

1 import socket
2 import _thread
3
4 def Traiter_Connexion(connexion_avec_client, adresse_client):
5     MessageRec=""
6
7     print( "Connexion de la machine = ", adresse_client)
8     try:
9         while(MessageRec.strip().lower()!="fin"):
10             MessageRec=connexion_avec_client.recv(1024)
11             MessageRec=MessageRec
12             print( "Client" ,adresse_client," a dit :",MessageRec.decode())
13     except:
14         print( "Deconnexion")
15     print( "Deconnexion de :",adresse_client)
16     try:
17         connexion_avec_client.close()
18     except:
19         pass
20
21 SocketServeur = socket.socket()
22 host = socket.gethostname()
23 port = 9500
24 SocketServeur.bind(("127.0.0.1", port))
25
26 SocketServeur.listen(5)
27
28 print( "Lancement serveur")
29 while True:
30     ConnexionAUnClient, addrclient = SocketServeur.accept()
31     _thread.start_new_thread(Traiter_Connexion, (ConnexionAUnClient,addrclient))
32
33
34
35

```

Listing 3 Programme serveur python multi-thread

Proto	Adresse locale	Adresse distante	Etat
TCP	127.0.0.1:9500	127.0.0.1:62778	ESTABLISHED
TCP	127.0.0.1:9500	127.0.0.1:62779	ESTABLISHED
TCP	127.0.0.1:61558	127.0.0.1:61559	ESTABLISHED
TCP	127.0.0.1:61559	127.0.0.1:61558	ESTABLISHED
TCP	127.0.0.1:61562	127.0.0.1:62556	ESTABLISHED
TCP	127.0.0.1:61612	127.0.0.1:61613	ESTABLISHED
TCP	127.0.0.1:61613	127.0.0.1:61612	ESTABLISHED
TCP	127.0.0.1:61614	127.0.0.1:61615	ESTABLISHED
TCP	127.0.0.1:61615	127.0.0.1:61614	ESTABLISHED
TCP	127.0.0.1:62556	127.0.0.1:61562	ESTABLISHED
TCP	127.0.0.1:62768	127.0.0.1:62770	ESTABLISHED
TCP	127.0.0.1:62770	127.0.0.1:62768	ESTABLISHED
TCP	127.0.0.1:62778	127.0.0.1:9500	ESTABLISHED
TCP	127.0.0.1:62779	127.0.0.1:9500	ESTABLISHED

Exercice : Interopérabilité Client / Serveur

III

A noter que les programmes client/serveur ne doivent pas nécessairement être écrit dans le même langage. Le concept de la création de Socket pour écouter / connecter à un numéro de port est le même dans la plus part des langages. Pour illustrer ceci, lancez le code Serveur python avant de lancer le client Java suivant qui fonctionne d'une manière identique au client python :

```

1 /*
2  * To change this license header, choose License Headers in Project Properties.
3  * To change this template file, choose Tools | Templates
4  * and open the template in the editor.
5  */
6 package Test;
7
8 /**
9  *
10 * @author DVSR
11 */
12 import java.net.*;
13 import java.io.*;
14 import javax.swing.JOptionPane;
15
16 public class Client {
17     public static void main(String [] args){
18         String NomDuServeur="127.0.0.1";
19         int port=9500;
20         Socket ConnexionAUnServeur;
21
22         DataInputStream FluxDEntrer;
23         DataOutputStream FluxSortie;
24         String MessageATransmettre;
25
26         try {
27
28             System.out.println("Lancement de l'application Client "+InetAddress.
29 getLocalHost());
30             ConnexionAUnServeur = new Socket (InetAddress.getByName (NomDuServeur
31 ), port);
32             FluxSortie = new DataOutputStream(ConnexionAUnServeur.getOutputStream
33 ());
34             FluxDEntrer = new DataInputStream(ConnexionAUnServeur.getInputStream
35 ());
36             do{
37                 MessageATransmettre=JOptionPane.showInputDialog("Entrez le message
38 à transmettre:");

```

```
34         byte[] b= new byte[MessageATransmettre.length()];
35
36         int i=0;
37         for(char c:MessageATransmettre.toCharArray()){
38             b[i]=(byte)c;
39             i++;
40         }
41
42         FluxSortie.write(b);
43
44
45         }while(!MessageATransmettre.equalsIgnoreCase("Fin"));
46
47         System.out.println("Deconnexion");
48         ConnexionAUnServeur.close();
49     }
50     catch (Exception e) {
51         System.out.println("Erreur "+e);
52     }
53 }
54
55 }
```

Analyse du trafic avec Wireshark

IV

La réalisation de ce TP nécessite l'installation de Wireshark :

<https://www.wireshark.org/download.html>

Lors du lancement de Wireshark soyez sûr que le logiciel est lancé en mode administrateur .

Ouvrez Wireshark et sélectionnez l'interface Npcap Loopback (1). Ensuite, insérez le filtre de port [*tcp.port == 9500*] en haut (2) afin d'observer seulement les paquets destinés / générés par le port TCP 9500.

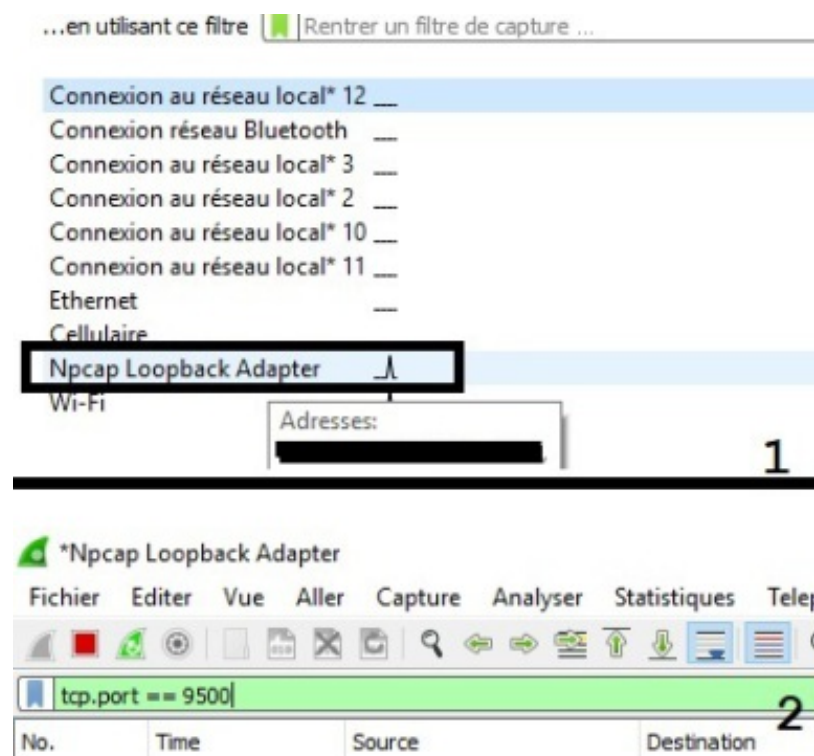


Figure 1. Interface loopback

Ci-dessous, les boutons permettant de a) d'arrêter la capture, b) redémarrer la capture, c) changer l'interface de capture.

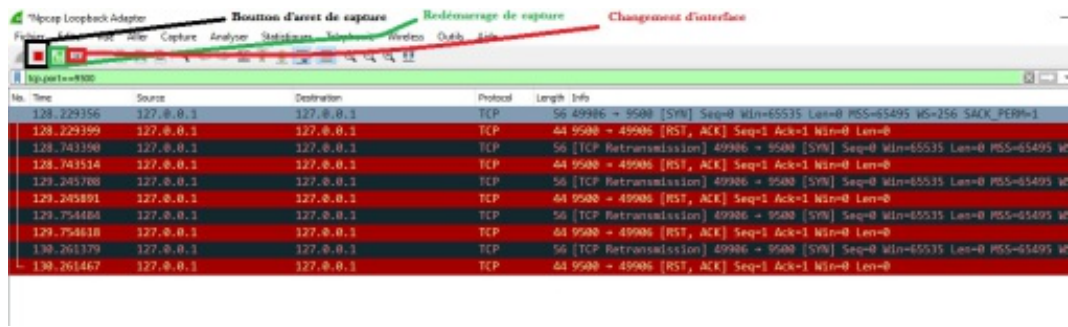


Figure 2. Interface de capture

Exercice : Three way handshake et flags

V

Question

1. Exécutez le client du premier exercice (Client.py) sans lancer le serveur et observez le résultat sur Wireshark. Utilisez le filtre approprié afin de trouver les segments TCP correspondants à numéro de port 9500. Quels sont les flags affichés dans la capture Wireshark ?
2. Exécutez le serveur (Serveur.py du TP 1) et ensuite le programme Client et observez le résultat sur Wireshark. Repérez les flags utilisés pour l'échange three-way-handshake dans la capture (les trois premiers segments échangés entre le client et serveur).
3. Transmettez un message du client vers le serveur et repérez le contenu de la transmission sur Wireshark (voir Figure 3).
4. Quels sont les flags utilisés dans le segment contenant les données envoyées par client ?
5. Quelles sont les options négociées initialement entre le serveur et client (sélectionnez les deux premiers échanges du three-way handshake afin de visualiser ces options)?
6. Transmettez un message "Fin" depuis le client vers le serveur afin de clôturer la connexion entre serveur et client. Repérez les flags utilisés pour clôturer la connexion.

8.602153	127.0.0.1	127.0.0.1	TCP	56 50059 → 9500 [SYN] Seq=0 Min=65535 Len=0 MSS=65495 WS=
8.602233	127.0.0.1	127.0.0.1	TCP	56 9500 → 50059 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MS
8.602296	127.0.0.1	127.0.0.1	TCP	44 50059 → 9500 [ACK] Seq=1 Ack=1 Win=2619648 Len=0
27.415797	127.0.0.1	127.0.0.1	TCP	66 50059 → 9500 [PSH, ACK] Seq=1 Ack=1 Min=2619648 Len=22
27.415958	127.0.0.1	127.0.0.1	TCP	44 9500 → 50059 [ACK] Seq=1 Ack=23 Win=2619648 Len=0

```

Frame 148: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 50059, Dst Port: 9500, Seq: 1, Ack: 1, Len: 22
Data (22 bytes)

```

```

0000  02 00 00 00 45 00 00 3e 8c de 40 00 00 06 00 00  ....E..> ..@.....
0010  7f 00 00 01 7f 00 00 01 c3 8b 25 1c 64 4f dc 3c  .f..00.01..f..00.01..c3..25.1c.64.4f.dc.3c
0020  5a e3 1b 21 50 18 27 f9 ad 81 00 00 4d 45 53 53  .a.e3.1b.21.50.18.27.f9.ad.81..00..4d.45.53.53
0030  11 47 45 20 56 45 52 53 20 20 6f 72 74 20 39 35  .1.47.45.20.56.45.52.53.20.20.6f.72.74.20.39.35
0040  10 30 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .10.30.....

```

Figure 3. Capture du contenu du message

Exercice : Déploiement d'une application réseau simple

VI

Question 1

Lancez le code Serveur et Client du premier exercice dans *deux machines appartenant au même réseau*. Dans le code client, vous devez remplacer l'adresse IP (variable host) par l'adresse IP de la machine où le serveur est situé. Il se peut que vous devez désactiver le pare-feu de la machine où le code serveur s'exécute.

Vous devez aussi changer l'adresse IP en paramètre de la méthode `SocketServeur.bind()` (code Serveur) par l'adresse IP de l'interface utilisée pour accéder au réseau local.

Si vous n'arrivez pas à connecter les deux applications, utilisez Wireshark dans la machine contenant le serveur afin de vérifier si le TWH est effectué en première place.

Lancez le code serveur suivant :

```
1 import socket
2 SocketServeur=socket.socket()
3 SocketServeur.bind(("0.0.0.0",50000))
4 SocketServeur.listen(4)
5 while True:
6     ConnexionAuClient, addr = SocketServeur.accept()
7     print(addr)
8     print(ConnexionAuClient.recv(1024))
9     ContenuReponse("<html><body><h1>FAKEHTMLSERVER INC &#xa9;</h1></body></html>")
10    ReponseServeur="HTTP/1.1 200 OK\r\nDate: Thu, 17 Jan 2019 13:18:42
    GMT\r\nServer: Apache/2.4.37\r\nAccept-Ranges: bytes\r\nContent-Length: %
    d\r\nContent-Type:HTML\r\n\r\n"% (len(ContenuReponse))
11    ConnexionAuClient.send(ReponseServeur.encode())
12    ConnexionAuClient.send(ContenuReponse.encode())
13    ConnexionAuClient.close()
```

- Celui-ci accepte les connexion entrantes (Ligne 6) ;
- Affiche l'adresse Socket du client (Ligne 7) ;
- Affiche le contenu reçu du client, requête HTTP dans ce cas (Ligne 8) ;
- Envoie les entêtes (Ligne 11) et le contenu HTTP (Ligne 12) ;

Question 2

- Accédez au serveur lancé à partir de votre navigateur (<http://localhost:50000>). Lisez l'affichage de la requête dans le serveur (Ligne 8) et repérez le contenu qui suit la séquence "User-Agent:".
- Accédez au serveur lancé à partir de smartphone ([http://\[VotreAdresseIPServeur\]:50000](http://[VotreAdresseIPServeur]:50000)) et repérez le contenu qui suit la séquence "User-Agent:".

